

Name : Vivek Chaudhari

Project : 12

Transitify : Scalable cloud-based bus booking system

tep-by-Step Implementation

◆ Step 1: Launch EC2 Instance

- Created EC2 instance (Ubuntu)
 - Configured Security Group:
 - Port 22 (SSH)
 - Port 80 (HTTP)
 - Port 5000 (App)
-

◆ Step 2: Connect to EC2

- Connected using SSH
 - Updated system and installed dependencies:
 - Python3
 - pip
 - Virtual environment
-

◆ Step 3: Setup Python Environment

- Created virtual environment
 - Installed required libraries:
 - Flask
 - MySQL Connector
-

◆ Step 4: Develop Backend Application

- Built Flask-based Bus Booking App
- Features:

- Book tickets
 - View bookings
 - Added HTML UI for better user experience
-

◆ **Step 5: Configure Amazon RDS**

- Created MySQL database
 - Enabled public access (for testing)
 - Created database and table:
 - bus_booking
 - bookings table
-

◆ **Step 6: Connect EC2 with RDS**

- Connected Flask app to RDS using endpoint
 - Verified database connectivity
 - Enabled inbound rule (Port 3306)
-

◆ **Step 7: Deploy Application**

- Ran Flask app on port 5000
 - Verified using EC2 public IP
-

◆ **Step 8: Create Load Balancer**

- Created Application Load Balancer
 - Configured:
 - Listener → HTTP (Port 80)
 - Target Group → EC2 instance (Port 5000)
-

◆ **Step 9: Configure Target Group**

- Registered EC2 instance
 - Configured health checks (/)
 - Verified status = **Healthy**
-

◆ Step 10: Configure Security Groups

- Allowed:
 - Port 80 (Load Balancer)
 - Port 5000 (EC2 from LB)
 - Ensured proper communication between components
-

◆ Step 11: Test Application

- Accessed via Load Balancer DNS
 - Verified:
 - Booking functionality
 - Data storage in RDS
 - Load balancer routing
-

Workflow (Simple Flow)

User → Load Balancer → EC2 (Flask App) → RDS (Database)

Key Features

- Cloud-based deployment
 - Scalable architecture
 - Real-time database integration
 - Load-balanced traffic handling
 - User-friendly interface
-



Final Summary

Developed and deployed a scalable Bus Booking Application using AWS services including EC2, RDS, and Application Load Balancer. Implemented a 3-tier architecture ensuring high availability, efficient traffic distribution, and real-time database interaction.

Snapshot :

bus-app-target

Actions

Details

arn:aws:elasticloadbalancing:ap-south-1:216153283160:targetgroup/bus-app-target/0cb98c507ba7dba0

Target type Instance	Protocol : Port HTTP: 5000	Protocol version HTTP1	VPC vpc-07abdf21cddb27f179
IP address type IPv4	Load balancer bus-app-lb		

1

Total targets

1

Healthy

0 Anomalous

0

Unhealthy

0

Unused

0

Initial

0

Draining

Distribution of targets by Availability Zone (AZ)

Colors in this table are recommendations. Please consult the Realtime console table below.

```
172.31.13.241 - - [24/Apr/2026 13:28:59] "GET / HTTP/1.1" 200 -
172.31.13.241 - - [24/Apr/2026 13:28:10] "GET / HTTP/1.1" 200 -
^C(myenv) ubuntu@ip-172-31-10-23:~$ nano app.py
(myenv) ubuntu@ip-172-31-10-23:~$ python app.py
Database & Table Ready
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.10.23:5000
Press CTRL+C to quit
172.31.13.241 - - [24/Apr/2026 13:28:59] "GET / HTTP/1.1" 200 -
172.31.32.104 - - [24/Apr/2026 13:28:59] "GET / HTTP/1.1" 200 -
172.31.13.241 - - [24/Apr/2026 13:28:59] "GET /favicon.ico HTTP/1.1" 404 -
172.31.13.241 - - [24/Apr/2026 13:29:10] "GET / HTTP/1.1" 200 -
172.31.13.241 - - [24/Apr/2026 13:29:22] "GET /book?name=vivek&source=dhule&destination=pune&seats=2 HTTP/1.1" 200 -
172.31.13.241 - - [24/Apr/2026 13:29:24] "GET / HTTP/1.1" 200 -
172.31.13.241 - - [24/Apr/2026 13:29:27] "GET /view HTTP/1.1" 200 -
172.31.32.104 - - [24/Apr/2026 13:29:29] "GET / HTTP/1.1" 200 -
172.31.13.241 - - [24/Apr/2026 13:29:32] "GET / HTTP/1.1" 200 -
172.31.13.241 - - [24/Apr/2026 13:29:40] "GET / HTTP/1.1" 200 -
172.31.32.104 - - [24/Apr/2026 13:29:59] "GET / HTTP/1.1" 200 -
172.31.13.241 - - [24/Apr/2026 13:30:10] "GET / HTTP/1.1" 200 -
172.31.32.104 - - [24/Apr/2026 13:30:29] "GET / HTTP/1.1" 200 -
```

bus-booking

Modify Actions

Summary

DB identifier bus-booking	Status Available	Role Instance	Engine MySQL Community	Recommendations
CPU 4.47%	Class db.t4g.micro	Current activity 1 Connections	Region & AZ ap-south-1a	

Connectivity & securityMonitoringLogs & eventsConfigurationZero-ETL integrationsMaintenance & backupsData migrationsTagsRecommendations

Connect using Info

Code snippets

CloudShell

Endpoints

Internet access gateway

Endpoint type

Programming language

Endpoint type

Connection steps

1 curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem

1 mysql -h bus-booking.c7iaqo6my8ct.ap-south-1-rds.amazonaws.com -P 3306 -u admin -p --ssl-mode=VERIFY_IDENTITY --ssl-ca=./global-bundle.pem

 **Bus Booking**

Book your ticket easily 🚀

Book Now

 **View Bookings**

✔ **Booking Successful**

Thank you vivek!

[Go Back](#)

 Booking Records

ID	Name	Source	Destination	Seats
1	vivek	dhule	pune	2
2	hitesh	dhule	shirpur	2
3	vivek	dhule	Mumbai	3
4	Bhumi	Shirpur	Pune	2

[← Back](#)